



12. Introduction to TensorFlow & Keras

"Know these two libraries for Deep Learning"

Previous Section:

 [11. Customizing Neural Networks Models](#)

Contents:

[1. TensorFlow and Keras](#)

[TensorFlow — The Engine](#)

[Keras — The Friendly Side of Deep Learning](#)

[2. TensorFlow Datasets](#)

[MNIST — Handwritten Digits](#)

[IMDB — Movie Reviews](#)

[Reuters — News Classification](#)

[CIFAR-10 — Image Classification](#)

[Fashion MNIST](#)

[3. TensorFlow and Keras: Building a Neural Network](#)

[3.1. Building a First Neural Network with Sequential Model](#)

[3.2. What is Happening Inside This Model?](#)

[3.3. Compile Step — How the Model Learns](#)

[3.4. Training the Model \(Fit Method\)](#)

[3.5. Evaluation — Testing the Model](#)

[3.6. Prediction — Making Decisions](#)

[3.7. Other Types of Models in Keras](#)

[3.8. Similarities with Scikit-Learn](#)

[3.9. Key Idea to Remember](#)

[4. TensorFlow & Keras Core Methods – Cheatsheet](#)

[4.1. Core Model Types in Keras](#)

[4.2. Common Layers in TensorFlow / Keras](#)

[4.3. Activation Functions](#)

[4.4. Optimizers](#)

[4.5. Loss Functions](#)

[4.6. Model Compilation \(Expanded\)](#)

[4.7. Training Methods](#)

[4.8. Callbacks](#)

[4.9. Model Utility Methods](#)

[4.11. TensorFlow Utility Functions](#)

[4.12. Mental Map](#)

[Summary](#)

[References](#)

Deep Learning sounds complicated at first. Neural Networks, tensors, backpropagation, GPUs..., everything feels heavy. And then we hear two names again and again: TensorFlow and Keras

At first glance, they look like two completely different libraries. Technically, they are. But today, they work together so closely that most people use them almost as one system.

Before TensorFlow became dominant, older frameworks like Theano were widely used for Deep Learning research. But over time, TensorFlow became one of the most important ecosystems in Machine Learning and Deep Learning.

TensorFlow is a free and open-source framework developed by Google. It is mainly used to build, train, and deploy Machine Learning and Deep Learning models. It supports many applications such as: Image Recognition; Natural Language Processing (NLP); Speech Recognition; Recommendation Systems; Time-Series Forecasting

And one of the biggest reasons for its popularity is this: It works almost everywhere. CPU; GPU; TPU. And sitting on top of TensorFlow is something much friendlier: Keras.

Keras is a high-level API designed to make Deep Learning easier to build and understand. Instead of writing massive amounts of low-level code, we can build powerful Neural Networks using only a few lines. So instead of struggling with implementation details, we can focus on the actual model.

And underneath everything in TensorFlow is one important idea: **Everything becomes tensors**. Images, text, audio, numbers — all of them must eventually become numerical data.

For example:

- A colored image becomes RGB pixel values
- A grayscale image becomes intensity values
- A sentence becomes numerical embeddings
- Audio becomes wave representations

And these numerical structures are stored as tensors. Think of tensors like generalized arrays:

- Scalar → single number
- Vector → 1D tensor
- Matrix → 2D tensor
- Higher dimensions → multi-dimensional tensors

So in many ways, Deep Learning is really about transforming data into tensors and learning patterns from them.

1. TensorFlow and Keras

Now let's finally explore them properly.

TensorFlow — The Engine

TensorFlow handles the heavy work behind Deep Learning. It performs: Tensor operations; Automatic differentiation; GPU acceleration; Model optimization; Neural Network computations

We usually import TensorFlow like this:

```
import tensorflow as tf
print(tf.__version__)
```

2.20.0

TensorFlow mainly works with tensors.

- **Creating Tensors**

```
import tensorflow as tf

a = tf.constant(5)
b = tf.constant(3)

print(a)
print(b)
```

tf.Tensor(5, shape=(), dtype=int32)

tf.Tensor(3, shape=(), dtype=int32)

We can also create vectors and matrices.

```
tensor = tf.constant([[1, 2],
                      [3, 4]])

print(tensor)
```

tf.Tensor(

[[1 2]

[3 4]], shape=(2, 2), dtype=int32)

- **Tensor Operations:** TensorFlow can perform mathematical operations directly on tensors.

Addition

```
a = tf.constant(5)
b = tf.constant(3)

result = a + b

print(result)
```

tf.Tensor(8, shape=(), dtype=int32)

Matrix Multiplication

```
matrix1 = tf.constant([[1, 2],
                       [3, 4]])

matrix2 = tf.constant([[2, 0],
                       [1, 2]])

result = tf.matmul(matrix1, matrix2)

print(result)
```

tf.Tensor(
[[4 4]
[10 8]], shape=(2, 2), dtype=int32)

```
tf.Tensor(  
[[ 4  4]  
[10  8]], shape=(2, 2), dtype=int32)
```

- **Automatic Differentiation with GradientTape:** One of the most important parts of Deep Learning is gradient computation. Instead of manually calculating derivatives, TensorFlow can automatically compute them using:

```
tf.GradientTape()
```

Example:

```
import tensorflow as tf

x = tf.Variable(3.0)

with tf.GradientTape() as tape:
    y = x ** 2

gradient = tape.gradient(y, x)
print(gradient)
```

tf.Tensor(6.0, shape=(), dtype=float32)

Mathematically, the derivative becomes: $\frac{dy}{dx} = 2x$. And when: $x = 3$. The gradient becomes $2(3) = 6$

This automatic differentiation system is one of the reasons TensorFlow became so powerful for Neural Networks.

Keras — The Friendly Side of Deep Learning

While TensorFlow is powerful, writing everything manually can become exhausting. That's where Keras comes in.

Keras is designed around one important philosophy: **Progressive disclosure of complexity**. Meaning that beginners can build models easily, while advanced users can still access low-level TensorFlow operations

Today, Keras is integrated directly inside TensorFlow:

```
from tensorflow import keras
```

Keras mainly provides: Layers; Models; Loss Functions; Optimizers; Metrics

Here is a very small Neural Network example:

```
from tensorflow import keras

model = keras.Sequential([keras.layers.Dense(8, activation
='relu'),
```

```

keras.layers.Dense(1)])

model.compile(optimizer='adam',
              loss='mse')

print(model)

```

Notice how little code we needed.

Without TensorFlow and Keras, building Neural Networks manually would require enormous amounts of mathematical implementation. That is why modern Deep Learning became much more accessible.

2. TensorFlow Datasets

Just like Scikit-learn provides built-in datasets, TensorFlow also includes many datasets for Deep Learning experiments.

Most datasets come from: `keras.datasets`. These datasets are extremely useful for beginners because we can focus on learning models instead of searching for data.

MNIST — Handwritten Digits

The most famous beginner dataset. It contains grayscale handwritten digits from 0–9.

```

from tensorflow.keras.datasets import mnist

(x_train, y_train), (x_test, y_test) = mnist.load_data()

print(x_train.shape)
print(y_train.shape)

```

```

(60000, 28, 28)
(60000,)

```

Each image is: 28 × 28 pixels; Grayscale; Digit classification

Example labels:

```
print(y_train[:10])
```

```
[5 0 4 1 9 2 1 3 1 4]
```

MNIST is often the first Deep Learning dataset people ever use.

IMDB — Movie Reviews

This dataset is used for Natural Language Processing. It contains movie reviews labeled as: Positive or Negative

```
from tensorflow.keras.datasets import imdb

(x_train, y_train), (x_test, y_test) = imdb.load_data(num_w
ords=10000)

print(len(x_train))
print(len(x_test))
```

This dataset is commonly used for: Sentiment Analysis; Text Classification; NLP experiments

Reuters — News Classification

The Reuters dataset contains news articles grouped into categories.


```
from tensorflow.keras.datasets import reuters

(x_train, y_train), (x_test, y_test) = reuters.load_data(num_words=10000)

print(len(x_train))
print(len(x_test))
```

8982

2246

Typical tasks include: Topic Classification; Multi-class Classification; NLP training

CIFAR-10 — Image Classification

CIFAR-10 contains colored images from 10 categories. Examples include: Airplanes; Cars; Cats; Dogs; Ships

```
from tensorflow.keras.datasets import cifar10

(x_train, y_train), (x_test, y_test) = cifar10.load_data()
print(x_train.shape)
```

(50000, 32, 32, 3)

Notice the final value: 3. That represents RGB channels. This dataset is commonly used for Computer Vision.

Fashion MNIST

Fashion MNIST is similar to MNIST, but instead of digits, it contains clothing images.

```
from tensorflow.keras.datasets import fashion_mnist

(x_train, y_train), (x_test, y_test) = fashion_mnist.load_data()
print(x_train.shape)
```

(60000, 28, 28)

Categories include: Shoes; Shirts; Bags; Coats.

This dataset is often considered a better beginner benchmark than MNIST because it is more difficult.

TensorFlow and Keras changed Deep Learning dramatically. Instead of manually implementing every equation and optimization step, we can now build complex Neural Networks using relatively small amounts of code. And as we continue deeper into Deep Learning, these two tools will become the foundation for almost everything we build.

3. TensorFlow and Keras: Building a Neural Network

Now that we understand the core of TensorFlow and Keras it is time to move from concepts to actual models.

Instead of writing complex mathematical operations from scratch, we define layers, connect them, and let the framework handle the learning process.

3.1. Building a First Neural Network with Sequential Model

The simplest way to build a neural network in Keras is using the Sequential model. It means: ***“Stack layers one after another, in a straight line”***. This works well for basic feedforward neural networks.

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense

# Create the model
model = Sequential([Dense(10, activation='relu', input_shape=(28, 28))])
```

```

e=(2,)),
        Dense(1, activation='sigmoid')])

# Compile the model
model.compile(optimizer='adam',
              loss='binary_crossentropy', metrics=['accuracy'])

# Show architecture
model.summary()

```

3.2. What is Happening Inside This Model?

Let's break it down step by step.

- **Input Layer:** Input shape = `(2,)`. This means each data point has 2 features (*implicitly defined, might need to reference the actual data*).
- **Hidden Layer:** `Dense(10, activation='relu')` : 10 neurons; Activation: ReLU; Purpose: learn intermediate patterns. ReLU introduces non-linearity, allowing the model to learn complex relationships instead of simple straight lines.
- **Output Layer:** `Dense(1, activation='sigmoid')` : 1 neuron; Sigmoid activation → Output: probability between 0 and 1. This is typically used for binary classification problems.

3.3. Compile Step — How the Model Learns

Before training, we must define how the model learns.

```

model.compile(optimizer='adam',
              loss='binary_crossentropy', metrics=['accuracy'])

```

- **Optimizer:** Adam → Controls how weights are updated during learning
- **Loss Function:** Binary Crossentropy → Measures how wrong the model is

- Metrics: Accuracy → Used for evaluation, not training

This step connects the model architecture to the learning strategy.

3.4. Training the Model (Fit Method)

Once compiled, we train the model using:

```
history = model.fit(X_train, y_train, epochs=20, batch_size=32,
                    validation_split=0.2)
```

- The model sees training data
- It adjusts weights using backpropagation
- It improves step by step over multiple epochs

The `history` variable stores training progress: Loss over time; Accuracy over time; Validation performance. This is extremely important for debugging and visualization.

3.5. Evaluation — Testing the Model

After training, we evaluate performance:

```
loss, accuracy = model.evaluate(X_test, y_test)
print(loss, accuracy)
```

This tells us: How well the model performs on unseen data. Whether it is overfitting or generalizing properly

3.6. Prediction — Making Decisions

Once trained, the model can predict new data:

```
predictions = model.predict(X_new)
```

The output depends on the task:

- Binary classification → probability
- Regression → numerical value

3.7. Other Types of Models in Keras

The Sequential model is only the beginning. Keras provides more flexible architectures:

- **Functional API:** Used when models have multiple inputs, models have multiple outputs, complex architectures are needed. It allows graph-like connections instead of simple stacking.
- **Model Subclassing:** Used when full control is needed, custom forward pass is required. This is closer to low-level deep learning design.

3.8. Similarities with Scikit-Learn

Keras is designed to feel familiar. Many methods look similar to Scikit-learn:

Concept	Keras	Scikit-learn
Training	<code>fit()</code>	<code>fit()</code>
Prediction	<code>predict()</code>	<code>predict()</code>
Evaluation	<code>evaluate()</code>	<code>score()</code>

This makes it easier to transition between classical ML and deep learning.

3.9. Key Idea to Remember

At this stage, the most important shift is this: We are no longer writing rules. Instead, we define architecture (layers), we define learning strategy (loss + optimizer), we let the model learn patterns from data

4. TensorFlow & Keras Core Methods – Cheatsheet

4.1. Core Model Types in Keras

Model Type	When to Use	Example
<code>Sequential</code>	Simple stacked layers	Basic feedforward networks
Functional API (<code>Model</code>)	Complex architectures	CNNs, multi-input/output models
Model Subclassing	Full customization	Research / custom training loops

4.2. Common Layers in TensorFlow / Keras

Layer	Purpose	Example Use
<code>Dense</code>	Fully connected layer	Standard neural networks
<code>Conv2D</code>	Feature extraction	Image models (CNNs)
<code>MaxPooling2D</code>	Downsampling	Reduce image size
<code>Flatten</code>	Matrix → vector	Bridge to Dense layers
<code>Dropout</code>	Prevent overfitting	Regularization
<code>BatchNormalization</code>	Stabilize training	Faster convergence
<code>Embedding</code>	Word vectors	NLP models
<code>LSTM</code>	Sequence modeling	Text / time series

4.3. Activation Functions

Activation	Meaning	When to Use
<code>relu</code>	Keeps positive values	Hidden layers (default)
<code>sigmoid</code>	Output 0–1 probability	Binary classification
<code>softmax</code>	Probability distribution	Multi-class classification
<code>tanh</code>	Range [-1, 1]	Older RNNs / hidden layers

4.4. Optimizers

Optimizers are NOT just strings—you can configure them as objects.

- **Basic usage (string shortcut):** `optimizer='adam'`
- **Advanced usage (recommended for real projects)**

Adam optimizer

```
from tensorflow.keras.optimizers import Adam

optimizer = Adam(learning_rate=0.001, beta_1=0.9,
                  beta_2=0.999, epsilon=1e-7)
```

Other optimizers

Optimizer	Idea
SGD	Classic gradient descent (can add momentum)
RMSprop	Good for RNNs
Adam	Most common (adaptive learning rate)

Why optimizer objects matter - You can tune training behavior:

- `learning_rate` → how fast model learns
- `beta_1`, `beta_2` → smoothing of gradients (Adam)
- `momentum` → speed + stability (SGD)

4.5. Loss Functions

Loss	When to Use
<code>binary_crossentropy</code>	Binary classification
<code>categorical_crossentropy</code>	Multi-class (one-hot labels)
<code>sparse_categorical_crossentropy</code>	Multi-class (integer labels)
<code>mse</code>	Regression problems

4.6. Model Compilation (Expanded)

```
model.compile(optimizer=Adam(learning_rate=0.001),
              loss='binary_crossentropy', metrics=['accuracy'])
```

Component	Purpose
optimizer	Controls weight updates
loss	Measures error
metrics	Monitoring only (not training)

4.7. Training Methods

Method	Purpose
<code>fit()</code>	Train model
<code>evaluate()</code>	Test performance
<code>predict()</code>	Generate predictions

`fit()` parameters (important additions)

```
model.fit(X_train, y_train, epochs=10, batch_size=32,
          validation_split=0.2, shuffle=True)
```

Parameter	Meaning
epochs	Full passes over dataset
batch_size	Samples per update
validation_split	Validation ratio
shuffle	Randomize training order

4.8. Callbacks

Callbacks control training dynamically.

- **EarlyStopping:** Stops training when model stops improving.

```
from tensorflow.keras.callbacks import EarlyStopping

early_stop = EarlyStopping(monitor='val_loss',
                           patience=3, restore_best_weights
                           =True)
```


- **ModelCheckpoint:** Saves best model automatically.

```
from tensorflow.keras.callbacks import ModelCheckpoint

checkpoint = ModelCheckpoint('best_model.h5', monitor='val_
loss',

                             save_best_only=True)
```

Why callbacks matter - They help you avoid overfitting, save best models automatically, stop wasting compute

4.9. Model Utility Methods

Method	Purpose
<code>model.summary()</code>	Show architecture
<code>get_weights()</code>	Extract weights
<code>set_weights()</code>	Load weights
<code>save()</code>	Save full model
<code>load_model()</code>	Reload model

4.10. Training History

```
history = model.fit(...)
```

Key	Meaning
<code>loss</code>	Training loss
<code>accuracy</code>	Training accuracy
<code>val_loss</code>	Validation loss
<code>val_accuracy</code>	Validation accuracy

4.11. TensorFlow Utility Functions

Function	Purpose
<code>tf.constant()</code>	Fixed tensor
<code>tf.Variable()</code>	Trainable parameter

Function	Purpose
<code>tf.random.normal()</code>	Random initialization
<code>tf.data.Dataset</code>	Efficient pipeline
<code>tf.reshape()</code>	Change shape

4.12. Mental Map

Every TensorFlow model follows:

1. **Build** → layers (Dense, Conv, LSTM)
2. **Configure learning** → optimizer + loss + metrics
3. **Train** → `fit()` + callbacks
4. **Evaluate** → `evaluate()`
5. **Use** → `predict()`

Summary

In this section, we introduced how neural networks are built using TensorFlow and Keras.

We learned that:

- Sequential models stack layers in order
- Dense layers form the core computation units
- Compile defines how learning happens
- Fit trains the model using data
- Evaluate tests performance on unseen data
- Predict applies the model to new inputs

And most importantly: TensorFlow does not “solve” problems directly. It builds systems that learn how to solve them.

Next, we will move from basic neural networks into more advanced architectures and training behavior, where learning becomes deeper, more unstable, and far more interesting.

References

<https://www.tensorflow.org/>

<https://www.geeksforgeeks.org/python/introduction-to-tensorflow/>

<https://keras.io/>

<https://www.geeksforgeeks.org/deep-learning/what-is-keras/>

Next Section:

 [13. Convolutional Neural Networks](#)